Comparing Detection Speed vs. Accuracy for DDoS Attacks Using Time-Window-Based Traffic Analysis

Mehedi Hasan Shanto
0112320044

Mahbub Uddin Ahmed
0112330085

Md Nazmul Haque
0112320157

Abstract—Fixed-time-window-based DDoS detection systems that inspect traffic must find the appropriate trade-off between how fast an attack is detected and the accuracy with which it is classified. In this study, this trade-off is assessed directly: the set of features, as well as the classifiers used to process them, remain unchanged while the size of the window changes (1s, 2s, 5s, 10s and 20s). Raw flow logs are first split into such windows, five very light features are computed for each window (number of packets, number of bytes, number of unique source IPs, ratio of packets with SYN flag and number of flows), windows are classified either as attack or benign, and two classifiers are assessed: Random Forest and Logistic Regression. The whole pipeline is first validated using a traffic generator system.However, on real CICDDoS2019 traffic, the dependence between performance metrics changes its trend for Random Forest: with respect to the UDPLag capture (725,165 flows), F1 decreases from 0.936 (at 1 s) to 0.725 (at 20 s) when effective latency increases from 1.0 s to 20 s, so the shortest window is the fastest and the most accurate one; similarly, with respect to the Syn capture, F1 decreases from 0.836 to 0.456. As to Logistic Regression, it is unstable at 1 s (F1 = 0.606 ± 0.195) and performs best at 5 – 10 s (F1 ≈ 0.83). The analysis at the phase level reveals that there are three consecutive attack phases within the UDPLag capture, and the portion of windows comprising both attack and benign traffic increases from 0% (at 1 s) to 84% (at 20 s), which can serve as an explanation.

Keywords—DDoS Detection, Network Traffic Analysis, Time Window Aggregation, Latency in Detection, Machine Learning, Random Forest, Intrusion Detection, CICDDoS2019

I. INTRODUCTION

A. Problem Statement and Motivation

DDoS attacks involve flooding the victim with traffic that is too much for the actual infrastructure to handle, and the problem of recognizing such attacks fast enough to allow their mitigation is an unresolved one. One method of DDoS attack recognition is to analyze the traffic data for a certain period of time, calculate some statistical parameters based on this data (packet count, bytes count, flag proportions, source IP diversity), and classify the window as either normal or malicious. The duration of the window is an implementation choice, having real-life implications – shorter window allows to make a decision faster, although based on less information, while a longer one provides more information but delays the decision-making process.

While very significant, the window size problem has always been perceived as a constant parameter instead of being considered as a variable that deserves its own investigation. This research topic has been selected due to two reasons: firstly, the tradeoff could be measured precisely and replicably using an open benchmark without any special equipment; secondly, the answer would have direct practical value – how many seconds of malicious traffic in exchange for some detection quality points?

B. Research Questions

RQ1. How do F1-Score and false alarm rate vary with respect to the increase in window size from 1 s to 20 s, respectively?

RQ2. What would be the effective detection latency for the same range of values? What percentage of attacks will be detected?

RQ3. Is the same directional relationship that is observed in the synthetic generator the same for real captures of CICDDoS2019?

RQ4. Which characteristics of the actual traffic account for the discrepancy?

C. Contributions

- A fully open pipeline (flow records → windows → five features → two classifiers → latency-aware evaluation) in which window size is the only variable.
- A fine-grained search sweep using (1, 2, 5, 10, 20 s) in which the variables of accuracy, false alarms, and latency are reported as a function of this single variable.
- Examples where the association found for the lab setting is inverted in reality for Random Forest, while Logistic Regression displays a distinct non-monotonic association.
- Phase-level and window-mixing analysis of the true capture explaining why large windows decrease in validity, along with the clear identification of validity threats.

D. Paper Organisation

Background information can be found in Section II, related work in Section III, methodology in Section IV, system implementation in Section V, experiment in Section VI, results in Section VII, discussion in Section VIII, and conclusion in Section IX.

II. BACKGROUND

A. DDoS Attacks and the CICDDoS2019 Dataset

In Sharafaldin et al. [1], the taxonomy distinguishes between reflection-based attacks in which the attacker remains concealed behind legitimate third-party servers that reflect and amplify the requests (such as DNS, LDAP, NetBIOS or MSSQL) and the exploitation-based attacks like SYN floods, UDP floods and UDP-Lag. The dataset is captured on two days. The first day called training day (12 January 2019) comprises twelve attack categories while the second day called testing day (11 March 2019) comprises seven attack categories namely PortScan, NetBIOS, LDAP, MSSQL, UDP, UDP-Lag and SYN. The flow records were extracted using CICFlowMeter and this software produces over 80 features per flow record [1]. This project utilizes the testing-day release (CSV-03-11), where each CSV file comprises flow records with a timestamp of initiation, source IP address, destination IP address, packets/bytes count and a label.

These two characteristics are defining features of this dataset. The first characteristic is that attack flows are much more numerous than benign flows in the dataset: 99.4 percent of flows in the UDPLag dataset are labeled as attacks. Second, since each file is named for an attack class, there could be different labels in the same file.

B. Time-Window Analysis and Detection Latency

Fixed windows break up time into successive, non-overlapping windows of length T where one decision is made per window. Attacks starting within window k cannot be detected until the window ends, which means that there are two factors involved in the latency: the latency until the window fills out (up to T) and the latency due to inference and feature extraction. As long as the inference and feature extraction processes are quick, the minimum latency will be set by T; a recent study shows that the data gathering portion dominates for fast models [3]. The sliding window approach, where the stride between windows is less than T, reduces the latency but increases computation and highly correlated data [10].

III. RELATED WORK

Benchmarks. CICDDoS2019 [1] is the commonly employed public dataset for benchmarking flow-based DDoS attacks detection. This dataset has been utilized in this research work.

Studies that vary the window. The most relevant prior work is ORACLE [2]. This work shifts the feature extraction to P4-based data plane, which reports statistics on the flows for the entire time window to an SDN controller at the end of the window. ORACLE tests windows of 5, 20, 40 and 60 s with Random Forest and K-nearest-neighbour classifiers on a replayed 90-minute trace. All eight models achieve more than 93% accuracy and show increasing accuracy with increasing window length; however, the increase is only modest (1.1% for KNN and 1.7% for RF) from the shortest to the longest window. Hence, the authors conclude that shorter windows should be used because of detection delay. A 2025 paper using multivariate time series models [3] also tests different window lengths and distinguishes latency in two parts: data collection and model computation. LUCID [5] uses time-window length as a hyper-parameter of its pre-processing stage; however, unlike other works, LUCID provides input to a convolutional network with arrays of per-flow packets.

Classical machine learning on CICDDoS2019. A number of works employ Random Forest, Logistic Regression and gradient boosting models on CICDDoS2019 dataset and achieve accuracies from 97% to 99% [4], [6]; however, they consider the traffic as a static table with already finished flows and do not pay any attention to the flow detection rate. A stream-based approach to traffic analysis is discussed in [8], whereas [9] surveys the field.

Lightweight statistical detection. Simple methods like the SYN-flood detector in [7] for SDN networks explain why a simple and low-cost set of features is used here.

Gap. The experiments of Table I either optimize for accuracy without regard for other factors, make a coarse adjustment of the window (to 60 seconds), or both in combination with more complicated deep learning models. No study holds all the other factors fixed while varying the window finely from 1 to 20 seconds.

TABLE I
COMPARISON WITH CLOSELY RELATED STUDIES

Study	Window range	Models	Latency
ORACLE [2]	5–60 s	RF, KNN	Discussed
Hernandez [3]	Varied	Deep time-series	Decomposed
LUCID [5]	Hyper-parameter	CNN	Proc. time
Windower [10]	Sliding	ML classifiers	Delay trade-off
[4], [6]	None (static)	RF, LR, CatBoost	Not studied
This work	**1–20 s**	**RF, LR**	**Measured**

IV. PROPOSED METHOD

A. Block Diagram of the Overall System

The pipeline is illustrated in Fig. 1. Raw CICDDoS2019 flow records are first loaded and preprocessed, then aggregated to time windows of a selected duration and five statistical features are extracted from each of them. The window table is then temporally divided into training and test segments and two different classifiers are trained and tested separately. The complete pipeline is executed for every window size (1s, 2s, 5s, 10s and 20s) to compare performance measures directly.

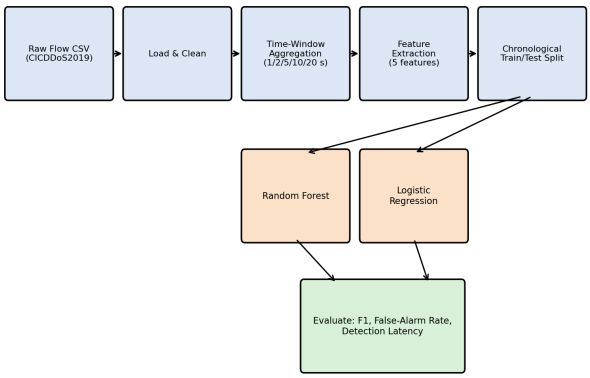


Fig. 1.End-to-end pipeline: raw flow records are windowed, transformed into five features per window, split chronologically into training and testing datasets, and used for training and evaluating two classifiers.

B. Data Loading and Cleaning

The CICFlowMeter table columns have some unwanted whitespace, therefore, the name of each column is cleaned first. The time stamp is parsed and those rows that have no time stamp value are dropped, the infinite values produced by the flow meter are set to be missing and those rows that lack an obligatory value are also dropped. A binary variable for an attack is created (Attack: any label except the BENIGN) and the data frame is ordered by time.

C. Windowing and Feature Extraction

With window length T and capture time t0, window k consists of $[t0 + (k - 1)T, t0 + kT)$. Let F(k) denote the set of flows that start at window k, $N(k) = |F(k)|$, and let pkts(f), bytes(f), syn(f), and src(f) denote the number of packets, bytes, SYN flags, and source addresses of a flow. The five features are:

- **Packet count** $P(k) = \Sigma$ pkts(f) over F(k);
- **Byte count** $B(k) = \Sigma$ bytes(f) over F(k);
- **Unique source IPs** $U(k) = |\{src(f) : f \text{ in } F(k)\}|$, an indicator of spoofed or distributed sources;
- **SYN ratio** $S(k) = \Sigma$ syn(f) / P(k), an indicator of SYN flooding (zero if P(k) = 0);
- **Flow count** N(k), a proxy for the arrival rate.

Windows with no flows are ignored, hence windows appearing consecutively in the table do not necessarily occur next to each other in time. The set is purposely kept minimal such that all features can be captured using just a few counters in real-time monitoring.

D. Window Labelling

An attack window is defined if at least 50 percent of the flows in the window are attack flows:

$y(k) = 1$ if $|\{f \in F(k): f \text{ is attack}\}| / N(k) \geq 0.5$, else 0(1)

This method of voting is straightforward to understand but produces some noise labels around the periphery of the attack region, and the amount of noise increases with T (Section VII-C).

E. Chronological Splitting and Repeated Evaluation

Since adjacent windows are correlated in time, splitting them randomly would lead to information leakage from the training to the test data set. Thus, the window table is split at a fraction of its length: the first windows form the training set and the latter ones the test set. The synthetic traffic traces are tested on ten independent samples. In case of the one real trace captured, a different split is used instead (50/50, 60/40, 70/30, and 80/20), resulting in four tests for each window size. If any side of a split contains only one class or less than five windows, the split is not used.

F. Classifiers

Two scikit-learn [11] classifiers learn using the same features and split. The first classifier is Random Forest [12], which uses 100 trees with balanced class weight, while the second classifier is logistic regression with balanced class weight applied to normalised features (maximum of 1000 iterations). The seed of both the classifiers is set to 42. Table II outlines the setup. Two models are selected because one represents non-linear and the other linear classifier; no deep learning and GPUs are used.

TABLE II
EXPERIMENTAL CONFIGURATION

Item	Setting
Window sizes T	1, 2, 5, 10, 20 s (disjoint)
Features	Packets, bytes, unique src IPs, SYN ratio, flows
Window label	Attack if ≥ 50% attack flows
Split (real)	50/50, 60/40, 70/30, 80/20 chronological
Split (synthetic)	70/30, ten seeds (100–109), 900-s traces
Random Forest	100 trees, balanced weights, seed 42
Logistic Regression	Standardised, balanced, max_iter 1000
Skip rule	One class only in train/test, or test < 5 windows

G. Evaluation Metrics

Where TP, TN, FP and FN are evaluated per test window, accuracy, precision, recall, F1-score and FAR are provided, that is, the ratio of benign windows falsely marked:

$F1 = 2TP / (2TP + FP + FN)$(2)

$FAR = FP / (FP + TN)$(3)

Detection latency. The ground truth in the test portion defines an attack episode as a maximal streak of consecutive attack windows. For an episode that begins at index s, where the classifier alerts for the first time at index j, the latency for that particular episode would be

$L = (j - s + 1) \cdot T$(4)

since it includes the delay incurred until the first window closes. The measured latency is thus the average value of L across episodes, and the detection rate is the percentage of episodes with at least one flagged window. Since latency L cannot be less than the window size T, the latter acts as a

lower bound on the former, which explains the need to interpret accuracy and latency side by side.

V. IMPLEMENTED SYSTEM

A. Software Environment and Tools

The project is done using only software without any special hardware. It utilizes Python 3 with pandas and numpy to load and aggregate the data, scikit-learn to use classifiers and metrics, and matplotlib to plot the results graphs. The actual data used are UDPLag.csv, Syn.csv and UDP.csv datasets which were captured on the testing day of CICDDoS2019 [1].

B. Code Organisation

There are three modules in the implementation. **ddos_pipeline.py** holds load_and_clean(), build_windows(), chronological_split(), get_models(), detection_latency(), and evaluate(). It also has some utility functions for plotting purposes. run_experiment.py serves as the multi-run driver for the experiments (run_synthetic(), run_real(), evaluate_one_split()). This module has the pipeline functions imported but no model or data specific functions, therefore the same driver can be used to do synthetic runs (by varying the random seed) or real data runs (varying the split point) by using a single command line flag. **generate_synthetic_data.py** creates the synthetic flows.

C. Synthetic Traffic Generator

Prior to collecting the real data, a synthetic generator for CICDDoS2019 was created for testing purposes. Traces are of 900 seconds long, with a benign base rate of 45 flows/second, and jitter within each second. There are four benign flash crowds, which peak multipliers are in the range 6-9x to mimic legitimate spikes similar to volumetric attacks. There are three attacks, a strong SYN flood lasting for 180-300 seconds and growing gradually within 12 seconds, a low-rate SYN attack that lasts for 450-520 seconds and is of 35% intensity with some overlapping flow characteristics with benign traffic, and a strong UDP flood for 640-760 seconds. The start of each attack is ramped and not instant, which makes the first windows of an attack ambiguous.

D. Reproducibility

Any form of randomness is deterministic: the classifiers are based on a random seed value of 42 while the traces have been generated based on random seeds 100 to 109. Any experiment generates one table per window size, with a raw table per run and an average ± std table, plus the F1 scores, latencies, false positives and trade-offs as well.

E. Figures of the Implemented System

Since this is a data processing and learning machine as opposed to a tangible instrument, it has two sets of diagrams – pipeline diagram (Fig. 1) and result figures in Section VII.

VI. EXPERIMENTAL SETUP

A. The Real Captures

The primary capture, UDPLag.csv, contains 725,165 flow records in 28.7 minutes, including 4,068 benign flows (0.56%) and 721,097 attack flows. Despite the file name, the capture contains three types of attacks that happen sequentially (Table III and Fig. 2). The first 11 minutes contain a UDP flood starting with an extremely high intensity attack burst, followed by 15 minutes of a UDP-Lag attack at a very low rate, and then a very short SYN flood happens. There are some benign flows throughout. Thus, the nature of the attack traffic changes over time, which means that the attack traffic is non-stationary. The attack flows originate from a very limited number of addresses (two sources in total).

TABLE III
PHASE STRUCTURE OF UDPLAG.CSV

Label	Flows	Start–end (s)
UDP	112,475	0–674
UDPLag	1,873	680–1570
Syn	606,749	1577–1722
BENIGN	4,068	1–1714

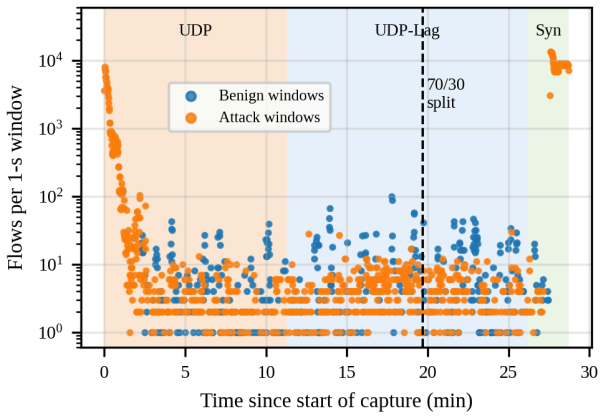


Fig. 2.Flows per 1-s interval during the UDPLag trace, highlighting the three stages of attacks and the cut-off point for the 70:30 ratio. Post initial attack flurry, most intervals carry just a few flows, either normal or malicious.

The second file, Syn.csv, was employed as a validation file and consisted of 164,334 flows by retaining only the benign flows while sub-sampling the attacks flows to make sure the file remains manageable. The third one, UDP.csv, has very few benign flows (0.08% of the file), which makes it inconclusive for certain window sizes.

B. Windows and Splits

The number of windows for each size is tabulated in Table IV. The number of windows reduces from 1,166 windows for a size of 1 s to 87 windows for a size of 20 s, and therefore large window tests rely on very few windows; when the split is 70/30, only 27 windows are in the test set for 20 s.

TABLE IV
UDPLAG WINDOWS PER SIZE (70/30 SPLIT)

Window	Total	Attack/benign	Train	Test
1 s	1,166	800/366	816	350
2 s	719	487/232	503	216
5 s	334	227/107	233	101
10 s	173	111/62	121	52
20 s	87	56/31	60	27

VII. RESULTS

A. Synthetic Data (Pipeline Validation)

Mean ± Standard Deviation for ten random traces are provided in Table V. For Random Forest, F1 increases from 0.982 at 1 s to 1.000 at 10 s, while the false positive rate decreases to zero. Logistic Regression also exhibits the same behavior but from a different initial point (0.964 at 1 s). The findings are consistent with the results of monotonic gain with respect to window size observed for ORACLE [2]. Figure 3 shows F1 versus effective latency: latency is traded off against accuracy on synthetic data sets.

TABLE V
SYNTHETIC DATA: MEAN ± STD OVER 10 RUNS

Win	RF F1	RF FAR	LR F1	LR FAR	Lat. RF/LR
1 s	0.982 ± 0.008	0.014 ± 0.009	0.964 ± 0.005	0.049 ± 0.009	1.2/1.7
2 s	0.989 ± 0.009	0.011 ± 0.012	0.976 ± 0.007	0.037 ± 0.014	2.0/2.2
5 s	0.996 ± 0.009	0.000 ± 0.000	0.971 ± 0.015	0.026 ± 0.014	6.0/8.0
10 s	1.000 ± 0.000	0.000 ± 0.000	1.000 ± 0.000	0.000 ± 0.000	10.0/10.0
20 s	1.000 ± 0.000	0.000 ± 0.000	1.000 ± 0.000	0.000 ± 0.000	20.0/20.0

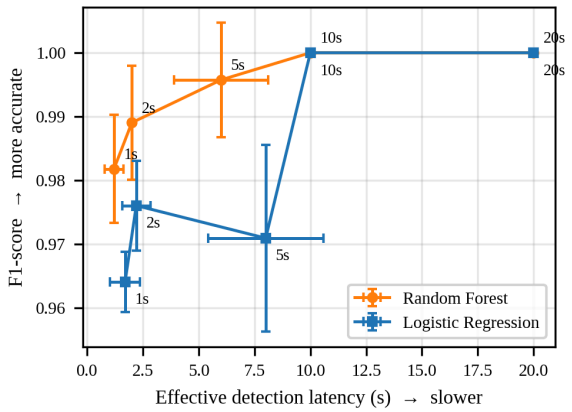


Fig. 3.Synthetic data: F1 score against the latency of detection for Random Forest and Logistic Regression using different window lengths.

B. Real Data: UDPLag Capture

Then, without any changes, the same pipeline was applied to UDPLag.csv. Results presented in Tables VI and VII are mean ± SD for four temporal splits and accuracy, precision and recall for each of them is provided in Table VIII. However, in case of Random Forest algorithm results are just the opposite: F1 measure decreases gradually from 0.936 at 1 second window to 0.725 at 20 seconds, false-alarm rate increases from 0.164 to 0.598, and the latency increases from 1.02 seconds to 20 seconds (Fig. 4 and Fig. 5). Variation across temporal splits is relatively low comparing to F1 drop of 0.21 and it means that decreasing in F1 measure is not caused by any particular temporal split. Switching from 10-second to 1-second windows leads to eleven times faster attacks detection (from 11.33 seconds to 1.02 seconds) and increases F1 by 0.14. Therefore, shorter window is better in all regards (Fig. 6). Accuracy is ordered in the same way from 91.3% (1 second) to 64.7% (20 seconds), and precision is decreased from 0.923 to 0.640 while recall remains between 0.78 and 0.95 (Table VIII).

TABLE VI
UDPLAG, RANDOM FOREST (MEAN ± STD, 4 SPLITS)

Win	F1	FAR	Latency (s)	Det. %
1 s	0.936 ± 0.009	0.164 ± 0.038	1.02 ± 0.01	96
2 s	0.905 ± 0.004	0.189 ± 0.028	2.16 ± 0.10	92
5 s	0.867 ± 0.012	0.311 ± 0.032	5.55 ± 0.40	95
10 s	0.796 ± 0.021	0.276 ± 0.021	11.33 ± 0.95	83
20 s	0.725 ± 0.043	0.598 ± 0.048	20.00 ± 0.00	81

TABLE VII
UDPLAG, LOGISTIC REGRESSION (MEAN ± STD, 4 SPLITS)

Win	F1	FAR	Latency (s)	Det. %
1 s	0.606 ± 0.195	0.391 ± 0.131	2.36 ± 1.21	57
2 s	0.756 ± 0.009	0.357 ± 0.025	2.62 ± 0.12	77
5 s	0.829 ± 0.014	0.305 ± 0.022	6.31 ± 0.16	88
10 s	0.836 ± 0.017	0.213 ± 0.031	10.00 ± 0.00	91
20 s	0.762 ± 0.035	0.504 ± 0.079	24.92 ± 1.26	100

TABLE VIII
UDPLAG: MEAN ACCURACY (%), PRECISION AND RECALL

Win	RF Acc	RF Prec	RF Rec	LR Acc	LR Prec	LR Rec
1 s	91.3	0.923	0.950	56.8	0.732	0.560
2 s	87.4	0.906	0.905	69.1	0.801	0.717
5 s	82.5	0.834	0.904	78.2	0.825	0.833
10 s	75.7	0.816	0.778	80.5	0.858	0.817
20 s	64.7	0.640	0.837	69.9	0.685	0.864

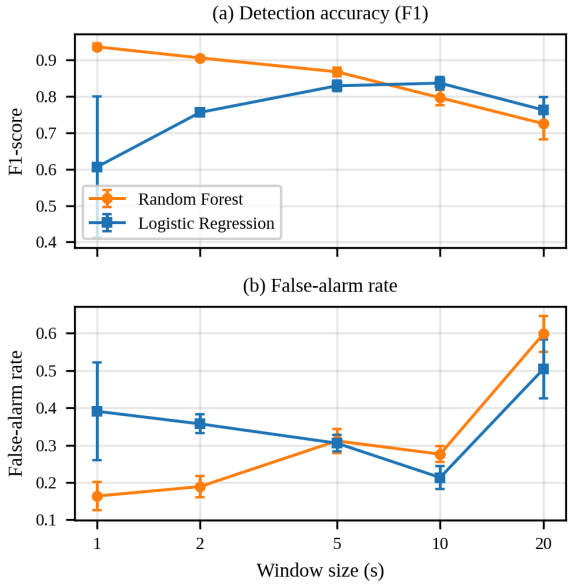


Fig. 4. UDPLag: (a) F1-score and (b) false-alarm rate vs. window size (mean ± std over four chronological splits).

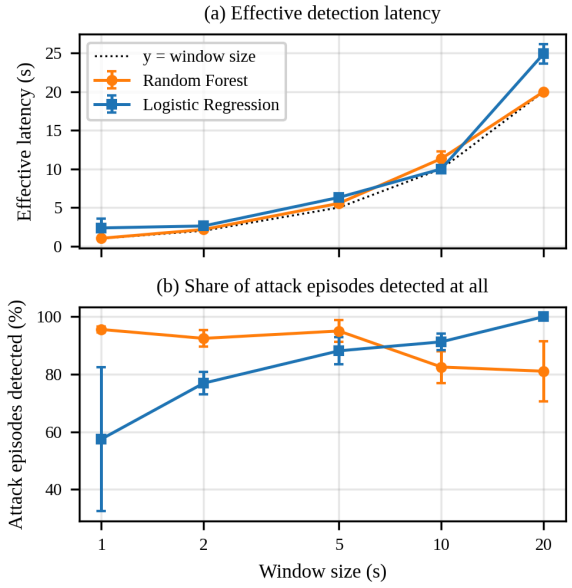


Fig. 5. UDPLag: (a) effective latency (dotted line: window size, the lower bound) and (b) share of attack episodes detected at all.

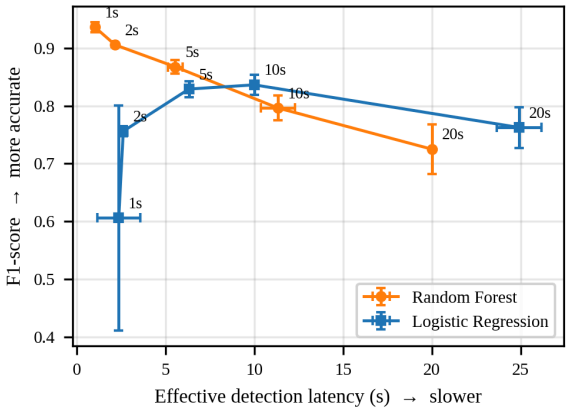


Fig. 6. UDPLag: F1-score vs. effective latency with error bars; points are labelled with the window size.

Logistic Regression exhibits different behavior (Table VII). It is unstable at 1 s (F1 0.606 ± 0.195, capturing only 57% of attacks), improves to 0.829 at 5 s and 0.836 at 10 s, then decreases to 0.762 at 20 s, due to its higher latency (24.9 s) than the window size since some attacks are detected only in subsequent windows. At 10 s it just outperforms the Random Forest (0.836 vs. 0.796), but the difference is roughly two standard deviations on four overlapping windows, thus it is indicative rather than decisive. According to Table IX, Logistic Regression is unstable only at 1 s (F1 0.408 and 0.470 at 50/50 and 60/40 split sizes) and becomes stable when the model has more history in its training dataset (0.784 and 0.761), while Random Forest is stable at 1 s (0.923-0.944).

TABLE IX
F1 PER CHRONOLOGICAL SPLIT (TRAIN SHARE)

Win	RF 50	RF 60	RF 70	RF 80	LR 50	LR 60	LR 70	LR 80
1 s	0.940	0.923	0.944	0.936	0.408	0.470	0.784	0.761
2 s	0.908	0.907	0.900	0.907	0.759	0.750	0.748	0.767
5 s	0.883	0.870	0.859	0.857	0.847	0.832	0.822	0.815
10 s	0.818	0.800	0.767	0.800	0.852	0.824	0.820	0.850
20 s	0.767	0.723	0.743	0.667	0.807	0.773	0.743	0.727

C. Where Do the Errors Come From?

There are three types of analysis which help in explaining the reversal, and they are descriptive but not causal.

Mixed Windows. With the 1-second windows being the most granular benchmark, Figure 7 presents the percentage of larger windows containing attack and benign seconds. The figure rises from 16% for 2-second windows to 46%, 70%, and 84% for 5-, 10-, and 20-second windows, respectively, with 75% of attack-labeled 20-second windows having benign seconds. The effect of averaging attack and normal behavior in one feature vector becomes more prominent as the windows get longer. It appears to be especially evident during the long, sparse UDP-Lag phase (Figure 2).

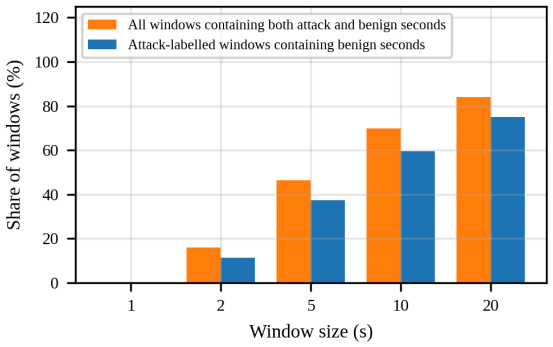


Fig. 7. Share of windows that mix attack and benign 1-s sub-windows, by window size.

Per-phase detection. The portion of correctly handled windows for the 70/30 split is given in Table X, with respect to each phase. The training set consists of the UDP flood and the beginning of UDP-Lag phases; the test set comprises the end of the UDP-Lag and SYN flood phases, which have never appeared in the training set. The SYN flood is very easy to detect for the two classifiers, whatever the window size is (90–100%), since it is very dense. However, the problem is with the rare UDP-Lag phase and the benign windows; the UDP-Lag recall of the Random Forest classifier is 93% at 1 s but just 59% at 10 s, and the benign windows are correctly handled 87% of the time at 1 s but only 42% at 20 s.

TABLE X
SHARE CORRECT BY PHASE (%), 70/30 SPLIT

Win	RF UDPLag	RF Syn	RF benign	LR UDPLag	LR Syn	LR benign
1 s	93	100	87	75	90	51
2 s	84	98	85	60	95	64
5 s	91	90	65	77	95	68
10 s	59	100	75	68	100	80
20 s	80	100	42	80	100	42

TABLE XI
CONFUSION COUNTS TN/FP/FN/TP, 70/30 SPLIT

Window	Random Forest	Logistic Regression
1 s	103/15/11/221	60/58/45/187
2 s	62/11/17/126	47/26/42/101
5 s	24/13/6/58	25/12/11/53
10 s	15/5/9/23	16/4/7/25
20 s	5/7/2/13	5/7/2/13

Feature importance. The Random Forest importances of 70/30 splits are depicted in Fig. 8. In 1-second splits, the byte and packet count features have the highest importances (0.37 and 0.32), whereas in 20-second splits the importance of the number of different source addresses is the highest (0.36). The importance of the SYN ratio feature is below 0.01 in all sizes; in this capture, the number of packets with SYN flags is zero for both classes, which means that there is no information in this feature. Attack windows have fewer unique source addresses than normal windows (average 2.2 and 2.9 for attack and benign windows, respectively at 1 second).

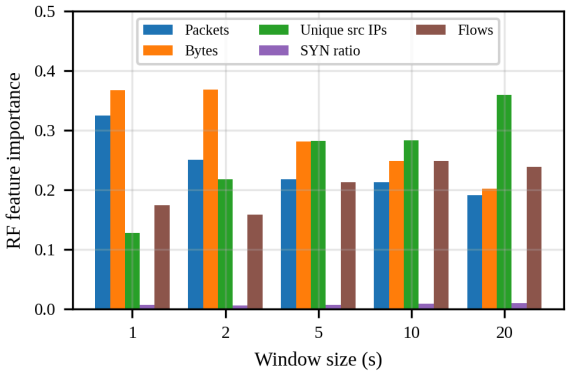


Fig. 8. Random Forest feature importance by window size (UDPLag, 70/30 split).

D. Cross-Check on Other Captures

This opposite trend was verified using Syn.csv. In the original tests, the Random Forest F1 score remained about 0.84-0.85 for 1-2 seconds, but then decreased to 0.456 by 20 seconds (Fig. 9). The reduction is greater than on UDPLag, with broad error margins at the larger sizes, and the false alarm rate remained small and constant (0.5-0.7%) regardless of the size. Therefore, the loss on this capture set occurs due to failure of detection rather than false positives. The Logistic Regression model performed much worse than the Random Forest one at all the sizes and detected no attack windows (F1 = 0.000) at 1 and 2 seconds. For UDP.csv there were no valid split at several sizes; the outcome is inconclusive.

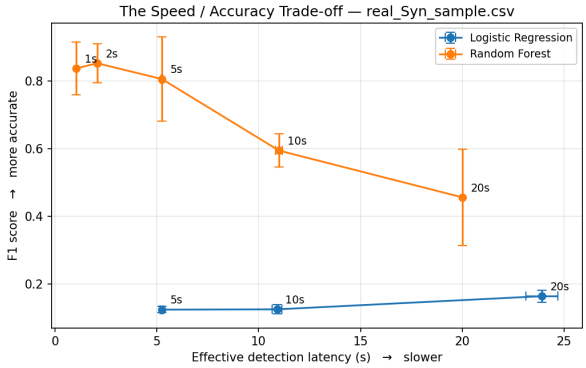


Fig. 9. Syn.csv (subsampled): the same reversed trend, more pronounced than on UDPLag.csv but with wide error bars at larger windows.

VIII. DISCUSSION

A. Interpretation

However, the key finding is that while the predicted trade-off between speed and accuracy of the expected classifier holds on synthetic data, for Random Forest on real data, the 1-s window classifier is both faster and more accurate than other window sizes. The analysis in Section

VII-C suggests that three factors work together, and explain the lack of speed–accuracy trade-off. Firstly, the real attack pattern is non-stationary, consisting of high rate burst, followed by long low rate period and finally flood attack at the end. Averaging such non-stationary attacks in long windows will wash out exactly the difference classifier uses to distinguish normal traffic from attack. Secondly, more noisy label windows are generated as T increases, which adds noise to the true signal itself. Thirdly, there are far fewer windows at large T (only 60 vs 816 at 20 s vs 1 s).

The attacks of the artificial generator are fixed within each attack phase, and their attacks deviate from normal traffic by the value of their SYN ratio and source variation, which are relevant in this case but not for the real UDPLag data. This explains why the same code produces different results and why lab-tested window size parameters should be checked against real-world traffic.

B. Practical Implications

With respect to Random Forest, window size of 1–2 seconds provides F1 score from 0.91 to 0.94 with latency of 1–2 seconds, while larger windows of 10–20 seconds result in higher cost of delay by factor of 10 and worse F1 score. Logistic Regression requires at least 5-10 seconds of data to produce a valid signal, and thus it is not as good as Random Forest regarding real-time processing. In other words, proper window size depends on the classifier, and optimal window size for one classifier should not be used for another. False alarms rate of 0.16 even at 1 second remains too high for production environment.

C. Threats to Validity

- **Overlapping splits.** The four split points have similar testing windows in common, hence the reported standard deviations may be lower than the actual uncertainties.
- **Single attack capture.** The primary finding relies on UDPLag and a subsetted Syn file; UDP.csv could not be tested. Another independent test is required.
- **Benign-class composition.** Only 0.56 percent of flows are benign. The many windows that are benign represent quiet times, making it somewhat easier compared to mixed production traffic.
- **Latency construct.** Latency is averaged only over episodes that are detected, and episodes are determined by window sizes (episodes split at 1 s and coalesce at 20 s), so values are not directly comparable between window sizes. Windows without activity are removed, so latency is measured in window intervals.
- **Flow-level aggregation.**The features are derived based on the flow records that have been collected and classified into time bins according to their start time.

D. Alternative Designs

Using a sliding window that overlaps (e.g., sliding by 0.5 seconds independent of the size of the window) could make our latency estimation smoother but at the expense of additional computation and correlations between training samples. Using a weighted or probabilistic window label would help alleviate any problems due to noisy labeling but at the expense of interpretability.

E. Complex Engineering Problem Attributes

The problem tackled in the project is firmly rooted in computer science and engineering (network security and machine learning). In-depth understanding of traffic analysis and learning was necessary (P1). Conflicting requirements exist where quick decision-making requires small windows and accurate estimates require many flows in each window and fine granularity results in many windows with little flow data per window (P2). The discovery of reversal of the real data trend needed extra analysis beyond the usual evaluation approach, which was phase analysis and window mixing analysis (P3) along with the consideration of non-stationarity and class imbalance particular to intrusion detection datasets (P4).

TABLE XII
COMPLEX ENGINEERING PROBLEM MAPPING

P1	P2	P3	P4	P5	P6	P7
✓	✓	✓	✓			✓

P1 depth of knowledge, P2 conflicting requirements, P3 depth of analysis, P4 familiarity of issues, P5 extent of applicable codes, P6 extent of stakeholders, P7 interdependence.

IX. CONCLUSION

In this project, a complete time-window-based DDoS detection pipeline was constructed and evaluated with respect to detection speed and accuracy on five window sizes using Random Forest and Logistic Regression algorithms. Larger windows produced better accuracy and less false positives on the synthetic dataset in line with previous literature. Shorter windows produced greater accuracy and speed on the CICDDoS2019 traffic for Random Forest in two independent captures: speed and accuracy was better for the windows of 1 second (UDPLag F1=0.936; latency 1.0s) than those of 20 seconds (F1=0.725; latency 20 s). For Logistic Regression, the algorithm was unstable at 1 second and performed best at 5–10 seconds; thus, the choice of an optimal window is specific to the model used. The capture consists of three phases of attacks that differ drastically in intensity, and the percentage of windows with attack and benign traffic increases from 0% to 84% depending on window size. This could provide an explanation for the results obtained. Recommendations about the optimal window size for non-stationary traffic are not applicable.

Further research should investigate additional CICDDoS2019 attack types using improved class balancing; analyze intermediate windows between 2 and 5 seconds where the Random Forest slope is most steep; limit the number of training windows in order to identify only the influence of the data size; perform analysis of

non-overlapping folds; and see if the results hold in case of the sliding window approach.

ACKNOWLEDGMENT

Acknowledgement goes to the Canadian Institute for Cybersecurity (CIC), University of New Brunswick, for open source of CICDDoS2019 dataset used in this research work.

REFERENCES

[1] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy," in *Proc. IEEE 53rd Int. Carnahan Conf. Security Technology (ICCST)*, Chennai, India, 2019.

[2] S. Gómez Macías, L. P. Gaspary, and J. F. Botero, "ORACLE: Collaboration of data and control planes to detect DDoS attacks," arXiv:2009.10798, 2020.

[3] D. V. Hernandez, Y.-K. Lai, and H. T. N. Ignatius, "Real-time DDoS detection in high-speed networks: A deep learning approach with multivariate time series," *Electronics*, vol. 14, art. 2673, 2025, doi: 10.3390/electronics14132673.

[4] "An efficient real time DDoS detection model using machine learning algorithms," arXiv:2501.14311, 2025.

[5] R. Doriguzzi-Corin, S. Millar, S. Scott-Hayward, J. Martínez-del-Rincón, and D. Siracusa, "LUCID: A practical, lightweight deep learning solution for DDoS attack detection," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 2, pp. 876–889, Jun. 2020, doi: 10.1109/TNSM.2020.2971776.

[6] "Optimized CatBoost machine learning (OCML) for DDoS detection in cloud virtual machines with time-series and adversarial robustness," PMC12808713.

[7] "Lightweight statistical approach towards TCP SYN flood DDoS attack detection and mitigation in SDN environment," Wiley/ResearchGate.

[8] "A flow is a stream of packets: A stream-structured data approach for DDoS detection," arXiv:2405.07232, 2024.

[9] "Detecting and mitigating DDoS attacks with AI: A survey," arXiv:2503.17867, 2025.

[10] P. Goldschmidt and J. Kučera, "Windower: Feature extraction for real-time DDoS detection using machine learning," in *Proc. NOMS 2024 – IEEE Network Operations and Management Symposium*, Seoul, 2024, doi: 10.1109/NOMS59830.2024.10575699.

[11] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[12] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.